

Exemplos do Nível ISA



UNIFEI

Universidade Federal de Itajubá

IMC – Instituto de Matemática e Computação

Prof. Carlos Minoru Tamaki

Torres de Hanói em linguagem de montagem do Core i7

Figura 5.44 Torres de Hanói para o Core i7.

```
.686 ; compile para processador da classe Core i7
.MODEL FLAT
PUBLIC _torres ; exporte 'torres'
EXTERN _printf:NEAR ; importe printf
.CODE
_torres: PUSH EBP ; salve EBP (ponteiro de quadro) e decremente ESP
        MOV EBP, ESP ; ajuste novo ponteiro de quadro acima de ESP
        CMP [EBP+8], 1 ; se (n == 1)
        JNE L1 ; desvie se n não for 1
        MOV EAX, [EBP+16] ; printf(" ...", i, j);
        PUSH EAX ; note que os parâmetros i, j e a cadeia
        MOV EAX, [EBP+12] ; de formato são passados para a pilha em ordem
        PUSH EAX ; inversa. Essa é a convenção de chamada em C
        PUSH OFFSET FLAT:format ; OFFSET FLAT significa endereço de formato
        CALL _printf ; chame printf
        ADD ESP, 12 ; retire parâmetros da pilha
        JMP Fim ; terminamos
```

L1:	MOV EAX, 6	; inicie $k = 6 - i - j$
	SUB EAX, [EBP+12]	; $EAX = 6 - i$
	SUB EAX, [EBP+16]	; $EAX = 6 - i - j$
	MOV [EBP+20], EAX	; $k = EAX$
	PUSH EAX	; inicie $\text{torres}(n - 1, i, k)$
	MOV EAX, [EBP+12]	; $EAX = i$
	PUSH EAX	; passe i
	MOV EAX, [EBP+8]	; $EAX = n$
	DEC EAX	; $EAX = n - 1$
	PUSH EAX	; passe $n - 1$
	CALL _torres	; chame $\text{torres}(n - 1, i, 6 - i - j)$
	ADD ESP, 12	; retire parâmetros da pilha
	MOV EAX, [EBP+16]	; inicie $\text{torres}(1, i, j)$
	PUSH EAX	; passe j
	MOV EAX, [EBP+12]	; $EAX = i$
	PUSH EAX	; passe i
	PUSH 1	; passe 1
	CALL _torres	; chame $\text{torres}(1, i, j)$

	ADD ESP, 12	; retire parâmetros da pilha
	MOV EAX, [EBP+12]	; inicie torres($n - 1, 6 - i - j, i$)
	PUSH EAX	; passe i
	MOV EAX, [EBP+20]	; EAX = k
	PUSH EAX	; passe k
	MOV EAX, [EBP+8]	; EAX = n
	DEC EAX	; EAX = $n - 1$
	PUSH EAX	; passe $n - 1$
	CALL _torres	; chame torres($n - 1, 6 - i - j, i$)
	ADD ESP, 12	; ajuste ponteiro de pilha
Fim:	LEAVE	; prepare para sair
	RET 0	; retorne ao chamador
.DATA		
format	DB "Mova disco de %d para %d\n"	; cadeia de formato
END		

Torres de Hanói em linguagem de montagem da CPU ARM do OMAP4430

Figura 5.45 Torres de Hanói para a ARM do OMAP4430.

```
#define Param0          r0
#define Param1          r1
#define Param2          r2
#define FormatPtr        r0
#define k                r7
#define n_minus_1       r5

        .text
torres:  push {r3, r4, r5, r6, r7, lr}           @ salve endereço de retorno e registradores mexidos
        mov r4, Param1
        mov r6, Param2
        cmp Param0, #1
        bne else                               @ (n == 1)?
        movw FormatPtr, #:lower16:format       @ se não, salte para sequência de código else
        movt FormatPtr, #:upper16:format
        bl printf                               @ carregue ponteiro da cadeia de formato
        pop {r3, r4, r5, r6, r7, pc}          @ mova para imprimir
```

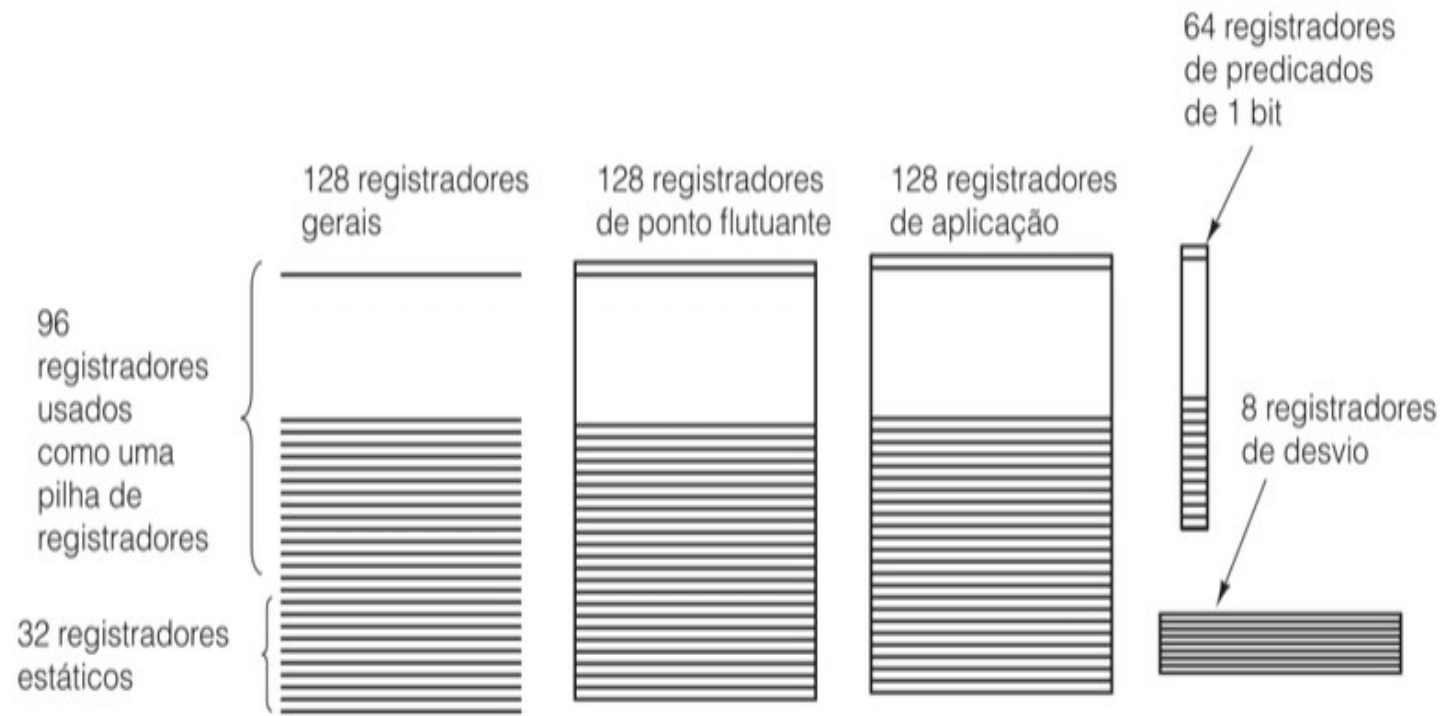
else:	rsb k, r1, #6	@ k = 6 - i - j
	subs k, k, r2	
	add n_minus_1, r0, #-1	@ calcule (n - 1) para chamada recursiva
	mov r0, n_minus_1	
	mov r2, k	
	bl torres	
	mov r0, #1	@ chame torres(n - 1, i, k)
	mov r1, r4	
	mov r2, r6	
	bl torres	
	mov r0, n_minus_1	@ chame torres(1, k, j)
	mov r1, k	
	mov r2, r6	
	bl torres	
	pop {r3, r4, r5, r6, r7, pc}	@ chame torres(n - 1, k, j)
		@ restaure registradores mexidos e retorne ao chamador
main:	.global main	
	push {lr}	
	mov Param0, #3	@ salve endereço de retorno do chamador
	mov Param1, #1	
	mov Param2, Param0	
	bl torres	
	pop {pc}	@ chame torres(3, 1, 3)
format:	.ascii "Mova um disco de %d para %d\n\0"	@ retire endereço de retorno, retorne ao chamador

A arquitetura IA-64 e o Itanium 2

- Core i7 possui a arquitetura EMT-64 que é uma extensão da ISA IA-32
- Juntamente com a HP, a Intel desenvolveu uma nova arquitetura a IA-64 totalmente de 64 bits uma ruptura total da arquitetura IA-32
- Problemas com ISA IA-32
 - Instruções de comprimento variável e vários formatos diferentes dificultando a decodificação
 - Base em 2 endereços com muitas referências a memória
 - Conjunto pequeno de registradores (4 a 6)

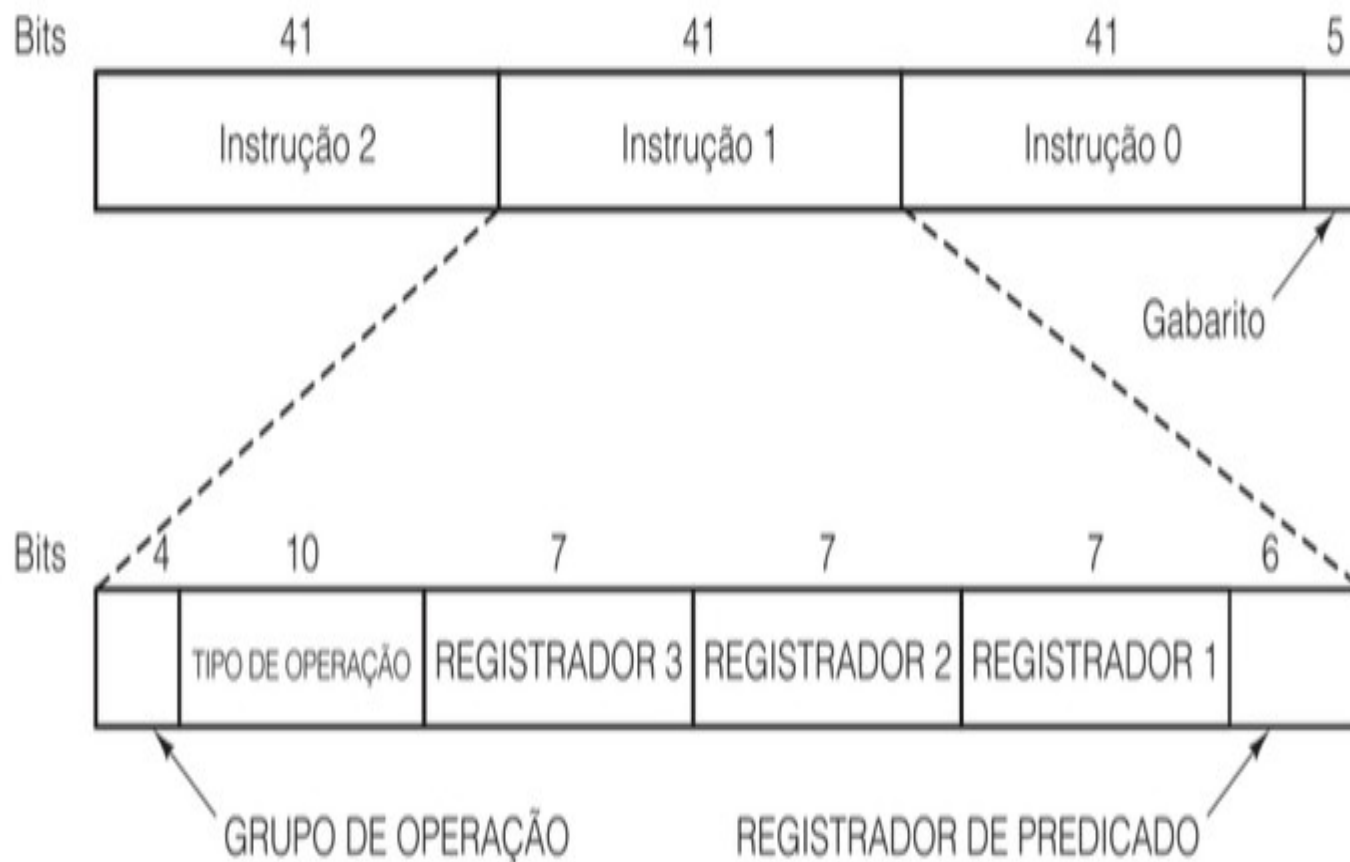
- O modelo IA-64: computação por instrução explicitamente paralela
- Redução de referência à memória

Figura 5.46 Registradores do Itanium 2.



- Escalonamento de instruções

Figura 5.47 O pacote IA-64 contém três instruções.



- Redução de desvios condicionais: predicação

Figura 5.48 (a) Declaração if. (b) Código genérico de montagem para (a). (c) Instrução condicional.

if (R1 == 0)

R2 = R3;

(a)

CMP R1,0

BNE L1

MOV R2,R3

L1:

(b)

CMOVZ R2,R3,R1

(c)

Figura 5.49 (a) Declaração if. (b) Código genérico de montagem para (a). (c) Execução condicional.

if (R1 == 0) {	CMP R1,0	CMOVZ R2,R3,R1
R2 = R3;	BNE L1	CMOVZ R4,R5,R1
R4 = R5;	MOV R2,R3	CMOVN R6,R7,R1
} else {	MOV R4,R5	CMOVN R8,R9,R1
R6 = R7;	BR L2	
R8 = R9;	L1: MOV R6,R7	
}	MOV R8,R9	
	L2:	
(a)	(b)	(c)

Figura 5.50 (a) Declaração if. (b) Código genérico de montagem para (a). (c) Execução predicada.

if (R1 == R2)	CMP R1,R2	CMPEQ R1,R2,P4
R3 = R4 + R5;	BNE L1	<P4> ADD R3,R4,R5
else	MOV R3,R4	<P5> SUB R6,R4,R5
R6 = R4 - R5	ADD R3,R5	
	BR L2	
	L1: MOV R6,R4	
	SUB R6,R5	
	L2:	
(a)	(b)	(c)

- Cargas especulativas
 - A IA-64 tem um modelo de paralelismo explícito que requer que o compilador decifre quais instruções podem ser executadas ao mesmo tempo sem conflitos e as agrupe em pacotes.
 - A predicação permite que as declarações em ambos os ramos de uma declaração *if* sejam fundidas em uma única cadeia, eliminando desvio condicional.

Bibliografia

- Organização Estruturada de Computadores – Andrew S. Tanenbaum, Tood Austin – 6ª Edição 2013 Prentice Hall
- Vários sites da internet